

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ  
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»  
(ДГТУ)

Факультет «Информатика и вычислительная техника»  
Кафедра «Кибербезопасность информационных систем»

### **Лабораторная работа № 1**

по дисциплине «Разработка защищённых Web-ресурсов»  
на тему «Создание Web-приложения с использованием фреймворка Flask для  
расчёта теоретической температуры горения»

Выполнил обучающийся гр. ВКБ53  
Ковалев Данил Петрович

Проверил: Ковальчик Р. В.

Ростов-на-Дону, 2026

## 1. Тема, цель и условие задания

Тема: создание начального Flask-приложения для технологических расчётов доменной плавки.

Цель: освоить создание изолированного Python-окружения, организацию проекта, маршрутизацию Flask и передачу HTML-шаблона браузеру.

По методическим указаниям требуется создать проект, установить Flask, зафиксировать зависимости, подготовить `index.html` и `tempr.html`, реализовать GET / и проверить домашнюю страницу в браузере.

Домашняя страница должна содержать заголовок и меню внутри `header/nav`, справочную таблицу внутри `main` и копирайт с автором внутри `footer`. Таблица переносится с рисунка 2 методических указаний.

По дополнительным требованиям проект создан только через `uv`, используется `src`-структура и чистая архитектура по образцу `answer_service`, зависимости внедряются через `Dishka`. Добавлен запуск в `Docker` через `Gunicorn`.

Граница первой работы: готовая домашняя страница и заготовка `/tempr`. В предоставленном задании форма и формула расчёта температуры отнесены ко второй части, поэтому численный расчёт здесь не реализуется.

### Краткая теория

Flask связывает HTTP-путь и метод с Python-обработчиком. Обработчик получает справочные данные и вызывает `render_template`; Jinja формирует HTML. Браузер получает HTML и отдельно загружает CSS. Чистая архитектура отделяет предметную модель от HTTP и сборки приложения.

## 2. Окружение и структура проекта

Проект и окружение `.venv` созданы менеджером `uv`. Использованы Python 3.14, Flask 3.1.3, Dishka 1.10.1 и Gunicorn 25.3.0. Точные версии и хеши фиксирует `uv.lock`; `requirements.txt` экспортируется из него для соответствия методичке.

```
uv init --app --package --name blast-furnace --python 3.14 \
  --build-backend hatchling --vcs none .
uv sync --frozen
uv run blast-furnace run --port 5001
uv export --frozen --no-dev --no-emit-project \
  --format requirements-txt --output-file requirements.txt
```

### Листинг 1 — создание, воспроизведение окружения и запуск

```
src/blast_furnace/
  app.py
  domain/reference.py
  application/get_blast_reference.py
  infrastructure/__init__.py
  presentation/http/
    routes.py
    templates/{base,index,tempr}.html
    static/styles.css
  setup/{app_factory,config,ioc,cli}.py
tests/{unit,integration}/
deploy/Dockerfile
docker-compose.yaml
docker-compose.dev.yaml
pyproject.toml, uv.lock, requirements.txt
justfile, .importlinter
```

### Листинг 2 — структура проекта

Внутренние слои `domain` и `application` не импортируют Flask, Dishka или внешние адаптеры. `Presentation` вызывает прикладной сценарий. `Setup` создаёт приложение и контейнер зависимостей. `Infrastructure` пока не содержит адаптеров: хранение данных и внешние системы в ЛР1 отсутствуют.

Границы слоёв контролируются четырьмя контрактами `import-linter`. Корневой `app.py` перенесён внутрь пакета по требованию к `src`-структуре. Проект размещён в существующем Git-репозитории DSTU\_VKB.

### 3. Сборка приложения и внедрение зависимостей

```
import atexit

from dishka import make_container
from dishka.integrations.flask import setup_dishka
from flask import Flask

from blast_furnace.presentation.http.routes import pages
from blast_furnace.setup.config import Settings
from blast_furnace.setup.ioc import ApplicationProvider

def create_app(settings: Settings | None = None) -> Flask:
    settings = settings or Settings.from_env()
    app = Flask("blast_furnace", static_folder=None)
    app.config.from_mapping(
        AUTHOR_NAME=settings.author_name,
        COPYRIGHT_YEAR=settings.copyright_year,
    )
    app.register_blueprint(pages)
    container = make_container(ApplicationProvider())
    setup_dishka(container=container, app=app)
    atexit.register(container.close)
    return app
```

#### Листинг 3 — фабрика Flask-приложения

```
from dishka import Provider, Scope, provide

from blast_furnace.application.get_blast_reference import GetBlastReference

class ApplicationProvider(Provider):
    reference_query = provide(GetBlastReference, scope=Scope.REQUEST)
```

#### Листинг 4 — регистрация прикладного сценария в Dishka

Фабрика регистрирует Blueprint и интеграцию Dishka. Для каждого HTTP-запроса создаётся REQUEST-область; обработчик получает GetBlastReference через FromDishka. Настройки автора и года передаются шаблонам через конфигурацию Flask.

```
"""WSGI entry point: gunicorn blast_furnace.app:app."""

from blast_furnace.setup.app_factory import create_app
from blast_furnace.setup.cli import main

app = create_app()

if __name__ == "__main__":
    main()
```

#### Листинг 5 — WSGI-точка входа пакета

## 4. Маршруты и справочная модель

```
from decimal import Decimal

from dishka.integrations.flask import FromDishka, inject
from flask import Blueprint, render_template

from blast_furnace.application.get_blast_reference import GetBlastReference

pages = Blueprint("pages", __name__, template_folder="templates", static_folder="static")

@pages.app_template_filter("decimal_ru")
def decimal_ru(value: Decimal) -> str:
    return str(value).replace(".", ",")

@pages.get("/")
@inject
def index(query: FromDishka[GetBlastReference]) -> str:
    return render_template("index.html", effects=query.execute())

@pages.get("/tempr")
def temperature() -> str:
    return render_template("tempr.html")
```

Листинг 6 — обработчики GET / и GET /tempr

Справочная модель `BlastParameterEffect` — неизменяемый `dataclass`. Поля: показатель, изменение параметра, изменение расхода кокса и изменение производительности. Для чисел применяется `Decimal`, а десятичная запятая формируется фильтром представления `decimal_ru`.

|                         |     |     |     |
|-------------------------|-----|-----|-----|
| Температура дутья       | 100 | 0,5 | 1,0 |
| Горячая прочность кокса | 1   | 1   | 1,6 |
| Расход природного газа  | 10  | 0,5 | 1,0 |

Листинг 7 — значения справочника с рисунка 2 методички

В исходной таблице не указаны единицы измерения и направления изменений. Они не добавляются предположительно, а сами справочные значения не используются как формула расчёта температуры.

## 5. Домашняя страница

Общий шаблон `base.html` содержит семантические блоки `header`, `nav`, `main` и `footer`. Шаблон `index.html` расширяет его и выводит строки справочника циклом Jinja. Заголовки таблицы имеют `scope`, таблица — `caption`. На узком экране прокручивается только таблица.

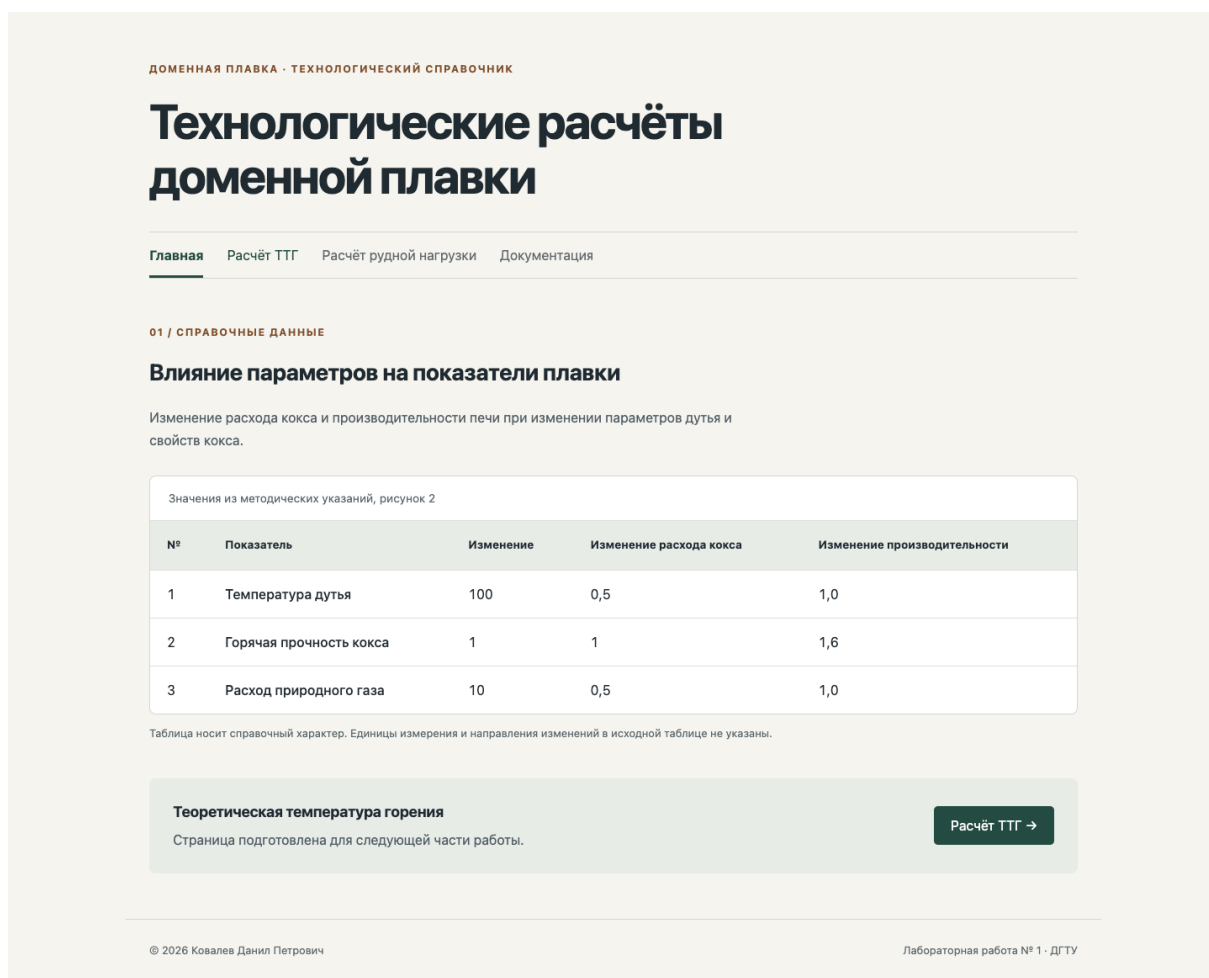


Рисунок 1 — главная страница работающего приложения

Страница открыта в Chrome по адресу `http://127.0.0.1:5001` из работающего Docker-контейнера. Внизу отображаются текущий год и автор: Ковалев Данил Петрович.

## 6. Страница расчёта температуры

Ссылка «Расчёт ТТГ» ведёт на /tempr. Страница явно сообщает, что форма и алгоритм будут добавлены во второй части, и содержит ссылку возврата к справочнику.

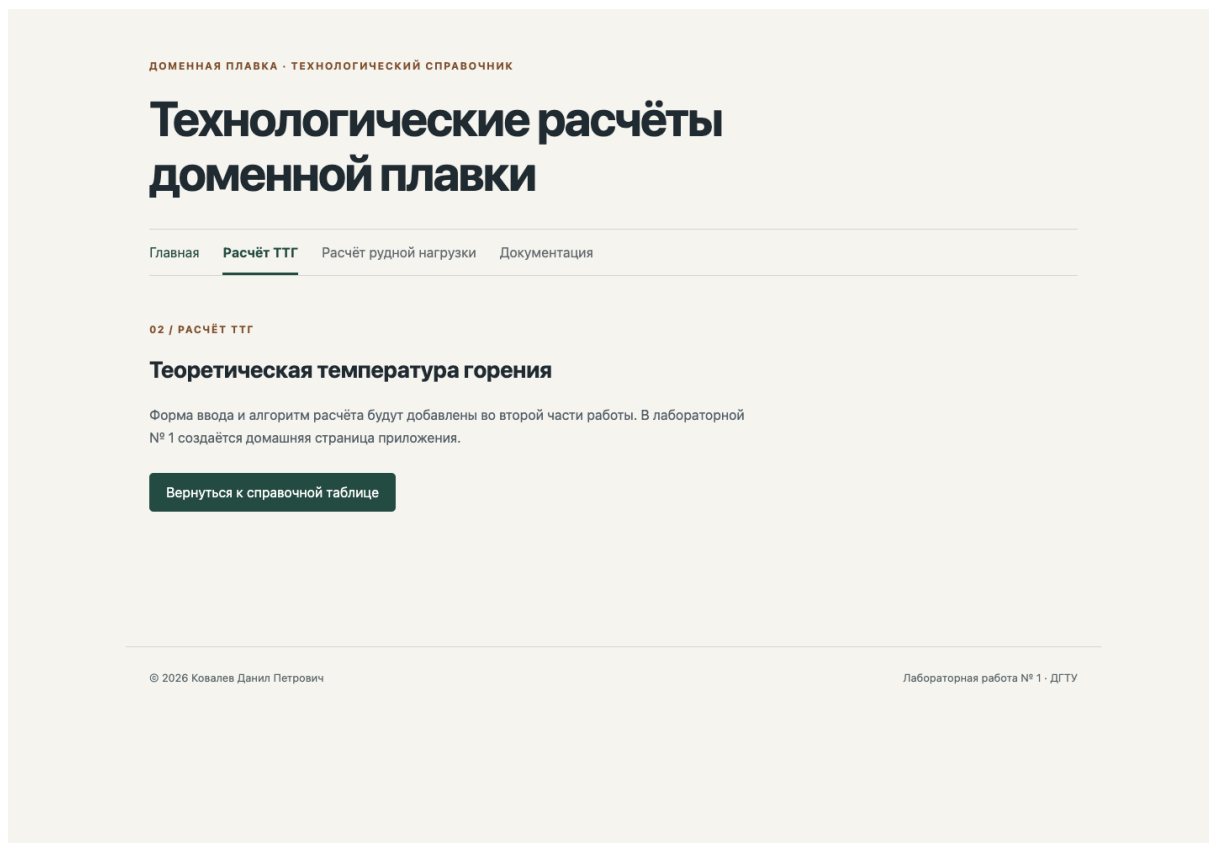


Рисунок 2 — заготовка страницы расчёта ТТГ

```
from blast_furnace.domain.reference import BLAST_PARAMETER_EFFECTS, BlastParameterEffect

class GetBlastReference:
    """Получить справочные данные для домашней страницы."""

    def execute(self) -> tuple[BlastParameterEffect, ...]:
        return BLAST_PARAMETER_EFFECTS
```

Листинг 8 — прикладной сценарий чтения справочника

Автоматическая браузерная проверка дополнительно выполнена при ширине 390 пикселей: страница не выходит за пределы окна. Снимок мобильной версии сохранён в docs/screenshots/index-mobile.png.

## 7. Запуск в Docker

Образ собирается в несколько стадий. В builder uv устанавливает зависимости по lock-файлу и пакет с HTML/CSS. В runtime копируется готовое окружение без инструментов тестирования. Приложение запускается от пользователя app (UID 10001).

```
# syntax=docker/dockerfile:1
FROM ghcr.io/astral-sh/uv:0.10.9 AS uv
FROM python:3.14-slim-bookworm AS builder

COPY --from=uv /uv /usr/local/bin/uv
ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy UV_PYTHON_DOWNLOADS=never
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN --mount=type=cache,target=/root/.cache/uv \
    uv sync --locked --no-dev --no-install-project
COPY README.md ./
COPY src ./src
RUN --mount=type=cache,target=/root/.cache/uv \
    uv sync --locked --no-dev --no-editable

FROM python:3.14-slim-bookworm AS runtime
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1 PATH="/app/.venv/bin:$PATH"
WORKDIR /app
RUN useradd --create-home --uid 10001 app
COPY --from=builder --chown=app:app /app/.venv /app/.venv
USER app
EXPOSE 8080
CMD ["gunicorn", "--bind", "0.0.0.0:8080", "--workers", "2", "--access-logfile", "-", "--error-logfile", "-", "blast_furnace.app:app"]
```

### Листинг 9 — Dockerfile

```
docker compose up --build -d --wait
docker compose logs -f app
docker compose down
```

### Листинг 10 — запуск, журнал и остановка контейнера

Compose публикует порт 127.0.0.1:5001 → 8080. Работают два процесса Gunicorn; healthcheck запрашивает главную страницу. Файловая система контейнера доступна только для чтения, /tmp выделен отдельно. Для разработки применяется overlay docker-compose.dev.yaml с исходниками и явным --debug.

## 8. Проверка результата

Проверены форматирование и стиль Ruff, строгая типизация mypy, статический анализ Bandit, архитектурные контракты import-linter и функциональные тесты pytest. Проверки завершились успешно.

```
uv run ruff format --check .
uv run ruff check .
uv run mypy
uv run bandit -c pyproject.toml -r src
uv run lint-imports
uv run pytest --cov --cov-report=term-missing
```

Листинг 11 — команды проверки

| Проверка          | Результат                             |
|-------------------|---------------------------------------|
| pytest            | 15 тестов пройдено                    |
| Ruff и mypy       | Ошибок не обнаружено                  |
| Bandit            | Замечаний не обнаружено               |
| import-linter     | 4 контракта соблюдены                 |
| Docker / Gunicorn | Контейнер healthy; UID 10001          |
| Chrome            | Главная, переход /tempr, экран 390 px |

HTTP-тесты проверяют точное совпадение всех трёх строк таблицы с заданием, разметку и копирайт, доступность ссылок и CSS, заготовку /tempr, ответы 404 и 405, экранирование имени автора, изоляцию настроек и отсутствие debug по умолчанию.

Текстовый протокол проверок сохранён в docs/verification.txt, снимки страниц — в docs/screenshots. Полные исходные тексты находятся в src/blast\_furnace.

## 9. Ответы на контрольные вопросы

### 1. Что такое виртуальное окружение? Зачем requirements.txt?

Окружение изолирует зависимости проекта и предотвращает конфликты версий. requirements.txt фиксирует пакеты для переноса и анализа состава ПО. Здесь он экспортируется через uv из uv.lock.

### 2. Чем templates отличается от пакета services?

templates содержит HTML-файлы для Flask/Jinja, поэтому импорт Python и \_\_init\_\_.py не нужны. services содержит импортируемые Python-модули; \_\_init\_\_.py обозначает обычный пакет. В проекте используются пакеты domain и application.

### 3. Зачем выносить бизнес-логику в отдельный пакет?

Три причины: тестирование без HTTP-сервера; повторное использование алгоритма, например из CLI; независимость предметных правил от изменений шаблонов и фреймворка.

### 4. Как связываются URL и обработчик? Что делает render\_template?

Декоратор route или get связывает URL и HTTP-метод с функцией. render\_template находит шаблон в настроенном каталоге templates, подставляет данные средствами Jinja и возвращает HTML.

### 5. Что даёт debug=True? Почему встроенный сервер не промышленный?

Debug включает перезагрузку и подробную диагностику с интерактивным отладчиком. Встроенный сервер предназначен для разработки, а не для обеспечения промышленной нагрузки и устойчивости. Отладчик нельзя публиковать для посторонних. В обычном Docker-режиме используется Gunicorn без debug.

## 10. Вывод и использованные источники

Выполнена первая лабораторная работа: создан Python-проект с окружением uv, реализованы Flask-приложение, домашняя страница со справочной таблицей и страница-заготовка расчёта температуры. Обеспечены семантическая разметка, src-структура, разделение слоёв и внедрение зависимостей через Dishka.

Подготовлены воспроизводимая контейнерная сборка, запуск через Gunicorn и режим разработки. Работоспособность подтверждена автоматическими тестами, статическими проверками, healthcheck контейнера и открытием страниц в браузере. Проект готов к расширению в последующих лабораторных работах.

1. ДГТУ. Лабораторная работа № 1 «Создание Web-приложения с использованием фреймворка Flask для расчёта теоретической температуры горения». Ростов-на-Дону, 2026. Предоставленный PDF, с. 3–9.

2. Flask. Application Setup.  
<https://flask.palletsprojects.com/en/stable/tutorial/factory/>

3. Dishka. Flask integration.  
<https://dishka.readthedocs.io/en/stable/integrations/flask.html>

4. uv. Using uv in Docker.  
<https://docs.astral.sh/uv/guides/integration/docker/>

5. Flask. Gunicorn.  
<https://flask.palletsprojects.com/en/stable/deploying/gunicorn/>

Дата обращения к электронным источникам: 11.09.2026.